# How to document these projects for my resume

Document them as **business-impact projects**, not as "apps I built." Use this pattern for every bullet:

**Action verb + what you built + technology/method + scale or measurable result**

Avoid vague phrases such as "worked on," "responsible for," or "developed a chatbot." Strong AI résumé bullets name the problem, the system, the stack, and an outcome such as accuracy, time saved, volume handled, latency, or error reduction. [1] [2] [3]

## Resume format

Put these under your current reinsurance-company experience as **Key Projects** or merge them into 4–6 bullets under the role. Use present tense because they are current work.

Example structure:

```
AI Engineer | Reinsurance Company | Month Year–Present
- Built...
- Designed...
- Developed...
- Created...
```

If you want them more visible, create a separate section:

```
Selected AI Projects
1. Data Migration & Validation Platform — C#, .NET, SQL Server
2. Document RAG Assistant — .NET, Phi-4, RAG
3. AI Regression Testing Platform — AI evaluation, automation
```

## Project 1: Data migration app

Do not write:

> Developed an app to migrate CSV files into SQL Server.

Write it as a data-quality and automation solution:

> **Built a CSV-to-SQL Server data-migration application** using C#/.NET that automated ingestion across business modules, validated source files, loaded data into staging tables, and archived processed files, reducing manual migration effort and improving traceability.

Better, more detailed bullets:

- **Designed and developed a modular CSV-to-SQL Server data-ingestion application** in C#/.NET to migrate sales, policy, and other module-level files into staging tables, replacing manual data-loading steps.
- **Implemented file-validation, batch processing, error handling, and automated archiving**, ensuring only complete and valid files were migrated while preserving an audit trail of processed batches.
- **Created reconciliation checks for row counts and key data totals** between source CSVs and SQL Server tables, improving data accuracy and making migration failures easier to detect. [4] [5] [6]
- **Reduced manual data-migration time by X%** and processed **N files / N records per batch**, depending on the actual volume.

If you have not measured a result yet, measure it now: count files, records, modules supported, manual steps removed, and time taken before versus after. Do not invent numbers; use scope metrics if business impact is unavailable. [7] [8]

### Project 2: RAG chatbot

Do not write:

> Made a chatbot using Phi-4.

Write it as a document-intelligence and knowledge-retrieval system:

> **Developed a document-based RAG assistant** in .NET using a locally hosted Phi-4 LLM, enabling users to query internal documents and receive source-grounded answers with citations.

Stronger bullets:

- **Built a .NET-based Retrieval-Augmented Generation (RAG) assistant** using a locally hosted Phi-4 model to answer questions from internal documents, reducing time spent manually searching policy, process, and reference documents.
- **Designed the document-ingestion and retrieval pipeline**, including text extraction, chunking, embedding, vector search, prompt construction, and response generation.
- **Implemented source-grounded responses with document/page citations**, helping users verify answers and reducing the risk of unsupported or hallucinated responses. [9] [10]
- **Created an evaluation set of N questions with expected answers and source documents**, measuring answer correctness, retrieval quality, refusal behavior, and hallucination rate.
- **Added guardrails for out-of-scope questions, missing evidence, and sensitive-data handling**, ensuring the assistant provides a “not found in documents” response when adequate evidence is unavailable. [11] [9]

A polished version:

> **Developed a .NET RAG document assistant powered by a locally hosted Phi-4 LLM**, with document ingestion, vector retrieval, source citations, and guardrails; evaluated responses on an N-question test set for groundedness, retrieval accuracy, and hallucination rate.

### Project 3: AI regression testing app

This is likely your strongest differentiator, so describe it as an **AI quality and reliability platform**.

> **Building an AI-based regression-testing platform** to automatically evaluate application and LLM behavior across releases, detect regressions, and provide structured pass/fail reports for development and QA teams.

Stronger bullets:

- **Designing an AI-powered regression-testing platform** that generates, executes, and evaluates test cases across application workflows, reducing repetitive manual regression effort.
- **Building automated validation rules for functional, data, and UI/API checks**, including expected outputs, schema validation, response-time thresholds, and failure classification.
- **Developing LLM evaluation capabilities** to test prompt, model, retrieval, and workflow changes using deterministic checks, rubric-based scoring, and human-review workflows. [12] [13] [11]
- **Creating baseline-comparison reports** that highlight newly failing, newly passing, and unchanged test cases after each prompt, model, or application change.
- **Integrating test execution into CI/CD** to catch regressions before release and maintain a version-controlled library of known failure cases. [13] [14]

A polished version:

> **Building an AI regression-testing platform** that combines automated functional checks, LLM-as-judge evaluation, and baseline comparison to detect regressions across prompts, models, and application releases.

### Add metrics without inventing them

Use these categories and fill in real values:

| Metric type | Examples you can measure |
|---|---|
| Scale | Number of files, records, documents, modules, users, test cases |
| Time | Manual hours saved, processing time, migration duration, query response time |
| Quality | Extraction accuracy, retrieval hit rate, answer groundedness, hallucination rate |
| Reliability | Test pass rate, regression-detection rate, number of defects caught |
| Adoption | Teams using the tool, documents indexed, requests per week |

| Metric type | Examples you can measure |
|---|---|
| Cost | Manual effort avoided, infrastructure cost, cost per query |

A good bullet usually needs **one strong metric**, not several competing numbers. If no business metric exists, use scope: “processing 50+ CSV files per batch,” “indexing 500 documents,” or “evaluating 100 test cases.” [3] [15] [16]

### Skills section

Add only tools you can discuss confidently in an interview:

```
AI/ML: RAG, LLM applications, prompt engineering, AI evaluation, LLM guardrails,
document intelligence, computer vision (YOLO), regression testing

Languages & Frameworks: C#, .NET, Python, SQL, JavaScript/TypeScript

Data & Databases: SQL Server, ETL/data migration, data validation, CSV ingestion,
staging tables, reconciliation

AI Tools: Phi-4, local LLM inference, vector search/embeddings, RAG pipelines

Engineering: REST APIs, Git, CI/CD, automated testing, Docker (if used),
Azure (if used)
```

Only include Python, Docker, Azure, or cloud services if you have actually used them or are actively learning them with a project to show.

### One-line project summary

If your résumé has limited space, use these three compact bullets:

- **Built a C#/.NET CSV-to-SQL Server migration application** with staging-table loading, validation, reconciliation, error handling, and automated archiving across business modules.
- **Developed a .NET document-RAG assistant using a locally hosted Phi-4 LLM**, providing source-cited answers through document ingestion, embedding, vector retrieval, and guardrails.
- **Building an AI regression-testing platform** that combines automated functional checks, LLM-based evaluation, and baseline comparison to detect regressions across application and AI-model changes.

For interviews, prepare a longer STAR story for each: the manual problem, your design decisions, challenges such as data quality or hallucinations, and the measurable or operational improvement. STAR is best used to prepare the story; compress it into concise action-result résumé bullets. [8] [17] [1]

⁂

1. https://fonzi.ai/blog/star-method-resume

2. https://engineersofai.com/docs/break-into-ai/resume-portfolio/ai-resume-framework
3. https://www.sweresume.app/articles/ai-engineer-resume/
4. https://learn.microsoft.com/en-us/data-migration/sql-server/database/guide
5. https://learn.microsoft.com/en-us/sql/relational-databases/post-migration-validation-and-optimization-guide?view=sql-server-ver17
6. https://kanerika.com/blogs/data-migration-testing/
7. https://fonzi.ai/blog/ai-engineer-resume
8. https://www.sweresume.app/articles/star-method-resume/
9. https://arxiv.org/html/2407.07858v1
10. https://brollyacademy.com/generative-ai-resume/
11. https://www.langchain.com/resources/llm-testing
12. https://www.confident-ai.com/blog/llm-testing-in-2024-top-methods-and-strategies?3a0b03e1_page=11
13. https://pathubs.com/blog/llm-regression-testing
14. https://www.vervali.com/in/blog/ai-and-llm-application-testing-in-2026-the-definitive-guide/
15. https://www.techiecv.com/job-search-toolkit/resume-writing/writing-resume-bullet-points
16. https://www.monster.com/career-advice/resume/how-to-write-accomplishments-on-resume
17. https://rezoom.io/how-to-write-resume-bullet-points/
18. https://scale.jobs/resume-templates/technical-writer
19. https://www.techiecv.com/resume-guides/ai-engineer-resume
20. https://www.sweresume.app/articles/xyz-method-resume/
21. https://github.com/Paramchoudhary/ResumeSkills/blob/main/skills/tech-resume-optimizer/SKILL.md
22. https://pagedone.io/blog/resume-bullet-points/
23. https://juicedresume.com/resume-guide/ai-engineer